

나이브 베이즈(분류 알고리즘)

: 베이즈 정리를 바탕으로 임의의 데이터가 특정 클래스(레이블)에 속할 확률을 계산하여 분류하는 **조건부 확률 기반 머신러닝 분류 알고리즘**

: 데이터를 나이브(단순)하게 독립적인 사건으로 가정하고, 이 독립 사건들을 **베이즈 이론에 대입시켜** 가장 큰 확률값을 갖는 모수를 추정해내는 지도 학습 분류 모델

- **나이브** : 모든 사건(특징)이 상호 독립적이라는 가정하에 확률 계산을 단순화, 모든 변수(특징)들이 동등하다는 것을 의미

* 베이즈 정리(Bayes' rule)

: 사건 B가 발생함으로써 사건 A의 확률이 어떻게 변화하는지(사건의 확률을 다시 계산)를 표현한 정리
: 새로운 정보가 기존의 추론에 어떻게 영향을 미치는지를 나타냄

공식

$$P(A | B) = \frac{P(B|A) * P(A)}{P(B)}$$



- $P(A | B)$: 사후확률, 사건 B가 발생한 후 갱신된 사건 A의 확률
 - $P(A)$: 사전확률, 사건 B가 발생하기 전에 가지고 있던 사건 A의 확률
 - $P(B | A)$: 가능도(likelihood), 사건 A가 발생한 경우 사건 B의 확률
 - $P(B)$: 정규화 상수, 또는 증거(evidence), 확률의 크기 조정역할
- (이미 발생한 사건의 확률)

* 베이즈 정리(Bayes' rule) 증명

공식

$$P(A | B) = \frac{P(B|A) * P(A)}{P(B)}$$

조건부 확률 : 어떤 사건 B가 일어났을 때 사건 A가 일어날 확률

공식 유도 과정

$$1) P(A | B) = \frac{P(A \cap B)}{P(B)} \Rightarrow P(A \cap B) = P(A | B) * P(B), P(B) \neq 0$$

$$2) P(B | A) = \frac{P(B \cap A)}{P(A)} = \frac{P(A \cap B)}{P(A)} \Rightarrow P(A \cap B) = P(B | A) * P(A), P(A) \neq 0$$

$$3) P(A \cap B) = P(A | B) * P(B) = P(B | A) * P(A)$$

$$4) P(A | B) = \frac{P(B|A) * P(A)}{P(B)}$$

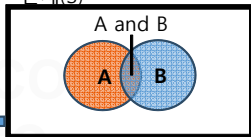


$$P(B | A) = \frac{P(A|B) * P(B)}{P(A)}$$

★
사건 A, B가 독립
이 아닐 경우

- $P(B | A)$: A 사건이 발생했을 때 사건 B가 발생할 확률
- $P(A | B)$: B 사건이 발생했을 때 사건 A가 발생할 확률

전체(S)



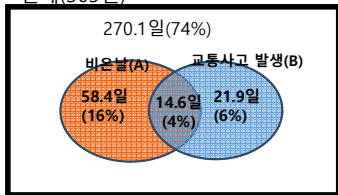
$P(A \cap B)$: 일반적인 A와 B의 교집합 확률
- 전체 S 중 A와 B가 겹치는 부분이 발생할 확률

$P(B | A)$: A 사건이 발생했을 때 B사건이 발생할 조건부 확률

- $P(B | A) = \frac{P(B \cap A)}{P(A)} = \frac{P(A \cap B)}{P(A)}$
- 사건 A 자체가 모수의 표본공간인 $P(A)$ 로 바뀜

	교통사고 발생일	교통사고 미발생일	계
비 온 날	14.6일 (4%)	58.4일 (16%)	73일 (20%)
비 안온 날	21.9일 (6%)	270.1일 (74%)	292일 (80%)
계	36.5일 (10%)	328.5일 (90%)	365일 (100%)

전체(365일)



$$P(B | A) = \frac{P(B \cap A)}{P(A)} = \frac{P(A \cap B)}{P(A)}, \quad P(A) \neq 0$$

$$P(A | B) = \frac{P(B \cap A)}{P(B)} = \frac{P(A \cap B)}{P(B)}, \quad P(B) \neq 0$$

$$P(B | A) \neq P(A | B)$$

(20%) ≠ (40%)

- 비가 오는 날 교통사고가 날 확률 :
단순 교집합 확률로 $(14.6/365) = 4\%$
- 비가 올 때 교통사고가 일어날 확률
조건부 확률로 $(14.6일 / 73일) = 20\%$
- 교통사고가 일어났을 때 비가 올 확률
조건부 확률로 $(14.6일 / 36.5일) = 40\%$



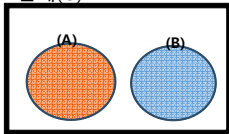
- **나이프** : 모든 사건(특징)이 상호 독립적이라는 가정하여 확률 계산을 단순화,
모든 변수(특징)들이 동등하다는 것을 의미

- 두 사건 A, B가 독립일 경우 조건부 확률의 단순화 (한 사건의 결과가 다른 사건에 영향을 주지 않을 때)
- : $P(A | B) = P(A)$, $P(B | A) = P(B)$
 - : $P(A, B) = P(A \cap B) = P(A) * P(B)$
 - : 결론, 독립일 경우 A 와 B가 동시에 발생할 확률은 그냥 두 사건 확률 곱으로 단순화 (뒷장 참조)

무조건부
독립

배반상태

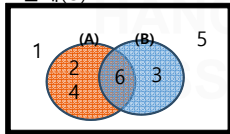
전체(U)



$\neq$

독립사건

전체(U)



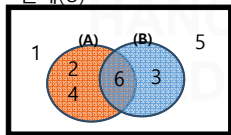
배반상태 : $A \cap B = 0$
공통분모(교집합)가 하나도 없는 상태

- $P(A)$: 주사위1개를 굴려서 2의 배수가 나올 확률
- $P(B)$: 3의 배수가 나올 확률

* 두 사건 이 독립인지 아닌지 애매한 경우 **독립 사건을 정의한 수식**으로 풀이

- P(A) : 주사위1개를 굴려서 2의 배수가 나올 확률
- P(B) : 3의 배수가 나올 확률

전체(U)



$$P(A \cap B) = \frac{1}{6}, \quad P(A) = \frac{3}{6}, \quad P(B) = \frac{2}{6} = \frac{1}{3}$$

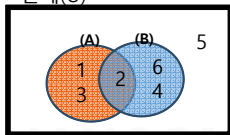
$$P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{\frac{1}{6}}{\frac{3}{6}} = \frac{1}{3} = P(B)$$

$$P(A \cap B) = \frac{1}{6}, \quad P(A) * P(B) = \frac{1}{2} * \frac{1}{3} = \frac{1}{6}$$

P(A ∩ B) = P(A) * P(B) 가 되어 사건 A 와 B는 독립

- P(A) : 주사위1개를 굴려 3이하의 수가 나올 확률
- P(B) : 짝수(2,4,6)가 나올 확률

전체(U)



$$P(A \cap B) = \frac{1}{6}, \quad P(A) = \frac{1}{2}, \quad P(B) = \frac{3}{6} = \frac{1}{2}$$

$$P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{\frac{1}{6}}{\frac{1}{2}} = \frac{1}{3} \neq P(B)$$

$$P(A \cap B) = \frac{1}{6}, \quad P(A) * P(B) = \frac{1}{2} * \frac{1}{2} = \frac{1}{4}$$

P(A ∩ B) ≠ P(A) * P(B) 가 되어 사건 A 와 B는 독립 되지 않음

조건부 독립 : 일반적인 독립과 달리 조건이 되는 확률변수가 존재

- A 와 B 사건은 C 사건 하에서는 서로 독립이다.

(사건 C 가 주어졌을때 A가 B에 대해서 또는 B가 A에 대해서 어떤 영향도 주지 않는다!)

- 두 사건 A, B가 별개의 확률변수 C에 **조건부 독립일 경우의 단순화**

: $P(A \cap B | C) = P(A | C) * P(B | C)$: C 사건이 발생한 경우에는 A,B는 서로 독립이 되는 확률

: $P(A | (B \cap C)) = P(A | C)$

: $P(B | (A \cap C)) = P(B | C)$

*** 나이브 베이즈 알고리즘 머신러닝에 적용**

$$P(A | B) = \frac{P(B|A) * P(A)}{P(B)}$$

$$P(\text{레이블} | \text{데이터 특징}) = P(\text{데이터 특징} | \text{레이블}) * P(\text{레이블}) / P(\text{데이터 특징})$$

* 나이브 베이즈 알고리즘 머신러닝에 적용

$$P(A | B) = \frac{P(B|A) * P(A)}{P(B)}$$

$$P(\text{레이블} | \text{데이터 특징}) = P(\text{데이터 특징} | \text{레이블}) * P(\text{레이블}) / P(\text{데이터 특징})$$

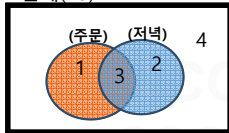
1개 특징일 경우

시간(특징)	치킨 주문(레이블)
오전	주문 안 함
오전	주문 안 함
점심	주문함
점심	주문 안 함
점심	주문 안 함
저녁	주문함
저녁	주문함
저녁	주문함
저녁	주문 안 함
저녁	주문 안 함

베이즈 이론 적용 :

손님이 저녁에 치킨을 주문할 확률 예측?

전체(10)



$$P(\text{주문} | \text{저녁}) = P(\text{저녁} | \text{주문}) * P(\text{주문}) / P(\text{저녁})$$

$$= (3/4) * (4/10) / (5/10)$$

$$= 0.6$$

= 60% 확률로 저녁에 치킨을 주문할 것으로 예측

2개 특징일 경우 : 입력 특징이 많은 경우 $P(y | x_1, x_2)$ 을 단순 계산하기 어렵기 때문에 각 특징이 **나아브하게 서로 독립적이라고 가정하여 계산의 복잡성을 낮춤** ==> **나이브 베이즈 알고리즘**



시간(특징1)	성인여부(특징2)	치킨 주문(레이블)
오전	성인	주문 안 함
오전	미성년자	주문 안 함
점심	성인	주문함
점심	미성년자	주문 안 함
점심	미성년자	주문 안 함
저녁	성인	주문함
저녁	성인	주문함
저녁	성인	주문함
저녁	성인	주문 안 함
저녁	미성년자	주문 안 함

나이브 베이즈 알고리즘 적용 :

시간, 성인여부 2가지 특성을 가지고
저녁에 성인 손님이 치킨을 주문할 확률 예측?



$$P(\text{주문} | \text{저녁, 성인})$$

A B

$$= P(A | B) = P(B | A) * P(A) / P(B)$$

$$= \frac{P(\text{저녁, 성인} | \text{주문}) * P(\text{주문})}{P(\text{저녁, 성인})}$$

조건부 독립 적용 :

$$P(A, B | C) = P(A | C) * P(B | C)$$

$$P(\text{저녁} | \text{주문}) * P(\text{성인} | \text{주문})$$

독립 적용:

$$P(A, B) = P(A) * P(B)$$

$$P(\text{저녁}) * P(\text{성인})$$



$$= P(\text{주문} | \text{저녁, 성인}) = P(\text{저녁} | \text{주문}) * P(\text{성인} | \text{주문}) * P(\text{주문}) / P(\text{저녁}) * P(\text{성인})$$

나이브 베이즈 알고리즘 적용 저녁에 성인이 치킨을 주문할 확률 ?

$$P(\text{주문} \mid \text{저녁, 성인}) = P(\text{저녁} \mid \text{주문}) * P(\text{성인} \mid \text{주문}) * P(\text{주문}) / \cancel{P(\text{저녁}) * P(\text{성인})}$$
$$= 0.3$$

주문함 중에 저녁일 확률

주문함 중에 성인일 확률

나이브 베이즈 알고리즘 적용 저녁에 성인이 치킨을 주문하지 않을 확률 ?

$$P(\text{주문 안함} \mid \text{저녁, 성인})$$
$$= P(\text{저녁} \mid \text{주문 안함}) * P(\text{성인} \mid \text{주문 안함}) * P(\text{주문 안함}) / \cancel{P(\text{저녁}) * P(\text{성인})}$$
$$= 0.066$$

공통분모식 제거

최종

: $P(\text{주문} \mid \text{저녁, 성인})$ 확률이 더 큼으로 저녁에 성인이 치킨을 주문할 것으로 분류

특징이 여러 개인 경우의 나이브 베이즈 공식화

예) $P(\text{주문} \mid \text{저녁}, \text{성인}) = P(\text{저녁} \mid \text{주문}) * P(\text{성인} \mid \text{주문}) * P(\text{주문}) / P(\text{저녁}) * P(\text{성인})$

$$P(y \mid x_1, \dots, x_n) = \frac{P(x_1 \mid y) P(x_2 \mid y) \dots P(x_n \mid y) P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

$$P(y \mid x_1, \dots, x_n) = \frac{P(y) \sum_{i=1}^n P(x_i \mid y)}{P(x_1)P(x_2)\dots P(x_n)}$$

공통분모 제거 $P(y \mid x_1, \dots, x_n) \propto P(y) \sum_{i=1}^n P(x_i \mid y)$

*** 가장 높은 수치의 레이블로 데이터 분류**

$$y = \operatorname{argmax} (P(y) \sum_{i=1}^n P(x_i \mid y))$$



주문과 주문안함에 대한
확률의 높은 수치로 분류

sklearn(사이킷런 : scikit-learn)

- 나이브 베이즈 알고리즘 장/단점

장점	단점
- 모든 데이터의 특징이 독립적인 사건이라는 나이브 가정에도 불구하고 높은 정확도 - 문서 분류 및 스팸 메일 분류에 강한 면모 - 나이브 가정에 의해 계산 속도가 다른 모델에 비해 상당히 빠름	- 모든 데이터 특징을 독립적인 사건이라 가정하는 것은 문서 분류에 적합할지 모르나 다른 분류 모델에는 제약이 될 수 있음 - 데이터가 가설과 다른 경우 정확도 떨어짐

1) 가우시안 나이브 베이즈 분류

- 특징들의 값들이 정규 분포(가우시안 분포)에 있다는 가정하에 조건부 확률을 계산
- 데이터의 특징이 이산적이지 않고 연속적인 성질이 있는 특징 데이터 분류에 적합

가우시안나이브베이즈 - 붓꽃데이터 시각화

```
import pandas as pd
from sklearn.datasets import load_iris # iris붓꽃 데이터 로드
from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB # 가우시안 나이브 베이즈 클래스 추가
from sklearn import metrics # confusion Matrix(혼동행렬) 성능 측정
from sklearn.metrics import accuracy_score # 정확도 평가

dataset = load_iris() # 붓꽃 데이터셋 로드
print(dataset)

iris_df = pd.DataFrame(dataset['data'], columns=dataset['feature_names'])
print(iris_df) # 150개 데이터
iris_df['target'] = dataset['target'] # target 레이블 추가
print(iris_df)
```

가우시안나이브베이즈 - 붓꽃데이터 시각화

```
# 시각화위해 숫자 형태의 레이블을 문자 형태로 변경
iris_df['target'] = iris_df['target'].map({0:'setosa',1:'versicolor',2:'virginica'})
print(iris_df)
# sepal length (cm) : 꽃받침 길이
# sepal width (cm) : 꽃받침 너비
# petal length (cm) : 꽃잎 길이
# petal width (cm) : 꽃잎 너비
# target : 붓꽃(iris) 종류 ==> setosa , versicolor , virginica

# 데이터 시각화
setosa_df = iris_df[iris_df['target'] == 'setosa']
versicolor_df = iris_df[iris_df['target'] == 'versicolor']
virginica_df = iris_df[iris_df['target'] == 'virginica']
#print(setosa_df)
```

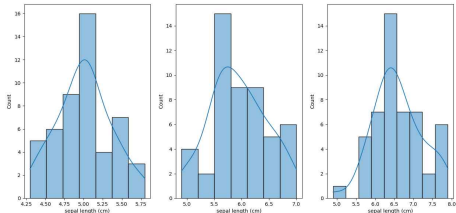
가우시안나이브베이지 - 붓꽃데이터 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
fig = plt.figure(figsize=(15,7))
ax1 = fig.add_subplot(1,3,1)
ax2 = fig.add_subplot(1,3,2)
ax3 = fig.add_subplot(1,3,3)
```

```
sns.histplot(data=setosa_df, x='sepal length (cm)', kde=True, ax=ax1)
sns.histplot(data=versicolor_df, x='sepal length (cm)', kde=True, ax=ax2)
sns.histplot(data=virginica_df, x='sepal length (cm)', kde=True, ax=ax3)
```

```
plt.show()
```



1-1) 가우시안나이브베이지 - 붓꽃분류 (GaussianNB)

```
import pandas as pd
from sklearn.datasets import load_iris # iris붓꽃 데이터 로드
from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB # 가우시안 나이브 베이지 클래스 추가
from sklearn import metrics # confusion Matrix(혼동행렬) 성능 측정
from sklearn.metrics import accuracy_score # 정확도 평가

dataset = load_iris() # 붓꽃 데이터셋 로드
print(dataset)

# 훈련데이터 / 테스트데이터 20% 비율로 분할
X_train, X_test, Y_train, Y_test = train_test_split(dataset['data'], dataset['target'],
                                                    test_size=0.2)

print(X_train.shape)

# 가우시안 나이브베이지 모델 학습
model = GaussianNB()
model.fit(X_train, Y_train)
```

1-1) 가우시안나이브베이지스 - 붓꽃분류 (GaussianNB)

모델 성능 테스트

```
predicted = model.predict(X_test)
print( 'Y_test : ', Y_test)
```

```
print( 'classes_:', model.classes_ ) # 모델의 정답에 대한 정보 확인
print( ' pred : ', predicted )
```

classes_ : [0 1 2]

pred : [2 0 0 2 1 1 1 1 0 2 2 2 0 1 2 1 0 0 1 0 1 2 1 0 1 2 0 2 1 2]

```
import numpy as np
np.set_printoptions(precision=6, suppress=True) # 지수표기법이 아닌 소수점 6자리까지 표현
print( 'predict_proba : \n', model.predict_proba(X_test))
```

```
metrics_report = metrics.classification_report(Y_test, predicted)
print(metrics_report)
print('accuracy : ', accuracy_score(Y_test, predicted))
```

predict_proba :

[[0.	0.000193	0.999807]
[1.	0.	0.]
[1.	0.	0.]
[0.	0.000001	0.999999]
[0.	0.999983	0.000017]

accuracy :
0.9666666666666667

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9
1	1.00	0.92	0.96	12
2	0.90	1.00	0.95	9
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

1-1) 가우시안나이브베이지스 - 붓꽃분류 (GaussianNB)

임의 데이터 예측 테스트

```
temp_predict = model.predict([[4.5, 3.3, 1.6, 0.2],  
                              [6.5, 3.7, 5.6, 2.2]])
```

```
print(temp_predict)
```

```
iris_class_name = ['setosa', 'versicolor', 'virginica']
```

```
import numpy as np
```

```
predicted_class_name = np.array(iris_class_name)
```

```
print(predicted_class_name[temp_predict]) # numpy fancy indexing
```

[0 2]

['setosa' 'virginica']

2) 베르누이 나이브베이지스 - 스팸분류 (BernoulliNB)

: 데이터의 특징이 0, 1로 표현됐을때 사용

: 스팸분류의 경우 특정 단어 출현 여부를 0, 1로 표현해서 사용

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score
```

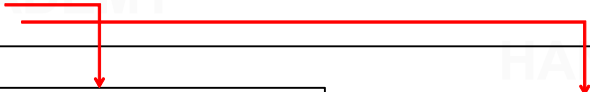
스팸 메일 분류를 위한 이메일 제목 과 스팸 레이블 준비

```
email_list = [
    {'email title': 'free game only today', 'spam':True},
    {'email title': 'cheapest flight deal', 'spam':True},
    {'email title': 'limited time offer only today only today','spam':True},
    {'email title': 'today meeting schedule', 'spam':False},
    {'email title': 'your flight schedule attached', 'spam':False},
    {'email title': 'your credit card statement', 'spam':False}
]
```

```
email_df = pd.DataFrame(email_list)
print(email_df)

# 분류를 위해 label ==> 숫자로 치환
email_df['spam'] = email_df['spam'].map({True:1,False:0})
print(email_df)

train_x = email_df['email title'] # 훈련데이터
train_y = email_df['spam'] # 타겟
print(train_x)
print(train_y)
```



```
0      free game only today
1      cheapest flight deal
2  limited time offer only today only today
3      today meeting schedule
4      your flight schedule attached
5      your credit card statement
Name: email title, dtype: object
```

```
0    1
1    1
2    1
3    0
4    0
5    0
Name: spam, dtype: int64
```

- 베르누이 나이브베이즈 입력 데이터는 고정된 크기의 벡터여야 함
- sklearn CountVectorizer() : 출현한 모든 단어 개수만큼의 크기를 가진 벡터를 생성하고 특정 데이터 안의 모든 단어를 고정 길이의 벡터로 표현
- CountVectorizer(), binary=True 설정 시
이메일 제목에 고정크기 벡터의 특정 단어가 출현할 경우 무조건 1, 단어가 출현하지 않을 경우 0 을 갖도록 설정

```
cv = CountVectorizer(binary=True)
train_x_cv = cv.fit_transform(train_x) # 훈련하고 변환
print(train_x_cv)

train_encoded = train_x_cv.toarray()
print(train_encoded)
```

```
[[0 0 0 0 0 0 1 1 0 0 0 1 0 0 0 1 0]
 [0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 1 0]
 [0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0]
 [1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 1]
 [0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 0 1]]
```

(0	6)	1	0번째 인덱스 문장
(0	7)	1	6, 7, 11, 15 index 위치
(0	11)	1	1로 채움
(0	15)	1	
(1, 2)	1		
(1, 5)	1		
(1, 4)	1		
(2, 11)	1		
(2, 15)	1		
(2, 8)	1		
(2, 14)	1		
(2, 10)	1		
(3, 15)	1		
(3, 9)	1		
(3, 12)	1		
(4, 5)	1		
(4, 12)	1		
(4, 16)	1		
(4, 0)	1		
(5, 16)	1		
(5, 3)	1		
(5, 1)	1		
(5, 13)	1		

```
print(cv.get_feature_names())
```

↓ (고정크기 벡터 의미)

고정된 벡터의 각 인덱스가 어떤 단어를 의미하는지 표현

['attached', 'card', 'cheapest', 'credit', 'deal', 'flight', 'free', 'game', 'limited', 'meeting', 'offer', 'only', 'schedule', 'statement', 'time', 'today', 'your']

- 문장에서 'attached' 출현 시 1
- 문장에서 'attached' 미 출현 시 0

- 문장에서 'game' 출현 시 1
- 문장에서 'game' 미 출현 시 0

- 문장에서 'today' 출현 시 1
- 문장에서 'today' 미 출현 시 0

문장) 'limited time offer only today only today' 경우

['attached', 'card', 'cheapest', 'credit', 'deal', 'flight', 'free', 'game', 'limited', 'meeting', 'offer', 'only', 'schedule', 'statement', 'time', 'today', 'your'] 이용

인코딩 결과

[0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 1 0]

CountVectorizer(binary=True) 설정으로
동일 단어 다수 출현 했지만 무조건 1로 표현

모델 학습

bnb = BernoulliNB()

```
train_y = train_y.astype('int')  
bnb.fit(train_encoded, train_y)
```

스팸으로 분류된 단어의 빈도수를 학습



모델 테스트를 위한 테스트 데이터 준비

```
test_email_list = [  
    {'email title': 'free flight offer', 'spam': True},  
    {'email title': 'hey traveler free flight deal', 'spam': True},  
    {'email title': 'limited free game offer', 'spam': True},  
    {'email title': 'today flight schedule', 'spam': False},  
    {'email title': 'your credit card information attached', 'spam': False},  
    {'email title': 'free credit card offer only today', 'spam': False}  
]
```

```
test_email_df = pd.DataFrame(test_email_list)  
test_email_df['spam'] = test_email_df['spam'].map({True:1,False:0})
```

```
test_x = test_email_df['email title']  
test_y = test_email_df['spam']
```

```
test_x_cv = cv.transform(test_x) # transform 만 해야함!!, 훈련데이터 가지고 이미 fit_transform()했음
test_encoded = test_x_cv.toarray()
print(test_encoded)
```

```
predicted = bnb.predict(test_encoded)
print(predicted) # [1 1 1 0 0 1]
print(test_y.values.ravel()) # [1 1 1 0 0 0]
print(bnb.classes_) # 0 : 정상 , 1 : 스팸,
print('accuracy : ', accuracy_score(test_y, predicted)) # accuracy : 0.8333333333333334
```

임의의 메일 제목 예측

```
temp_mail_cv = cv.transform(['last discount event of today',
                             'the payment document is attached and sent',
                             'company collaboration event free offer'])
print(temp_mail_cv.toarray())
predicted = bnb.predict(temp_mail_cv)
print(predicted) # [1 0 1] ==> [스팸, 정상, 스팸]
```

스무딩(smoothing) 기본 적용 : 학습데이터에 없던 데이터가 출현해도 빈도수에 1을 더해서
확률이 0이 되는 현상을 방지

스팸으로 분류된 단어의 출현이 많으면(조건부 확률) 스팸으로 분류

3) 다항분포 나이브베이즈 - 영화리뷰 분류 (MultinomialNB)

: 데이터의 특징이 출현 횟수 표현됐을때 사용

: 영화리뷰 분류의 경우 특정 단어 출현 회수(빈도수)를 표현해서 사용

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB # 다항분포 나이브베이즈
from sklearn.metrics import accuracy_score

# 영화 리뷰 데이터 준비
review_list = [
    {'movie_review': 'this is great great movie. I will watch again', 'type': 'positive'},
    {'movie_review': 'I like this movie', 'type': 'positive'},
    {'movie_review': 'amazing movie in this year', 'type': 'positive'},
    {'movie_review': 'cool my boyfriend also said the movie is cool', 'type': 'positive'},
    {'movie_review': 'awesome of the awesome movie ever', 'type': 'positive'},
    {'movie_review': 'shame I wasted money and time', 'type': 'negative'},
    {'movie_review': 'regret on this move. I will never never what movie from this director', 'type': 'negative'},
    {'movie_review': 'I do not like this movie', 'type': 'negative'},
    {'movie_review': 'I do not like actors in this movie', 'type': 'negative'},
    {'movie_review': 'boring boring sleeping movie', 'type': 'negative'},
]
```



```
movie_review_df = pd.DataFrame(review_list)
#print(movie_review_df)

movie_review_df['label'] = movie_review_df['type'].map({'positive':1,'negative':0})
#print(movie_review_df)

train_X = movie_review_df['movie_review']
train_Y = movie_review_df['label']
```

다항분포 나이브베이지 입력 ==> 고정된 크기의 벡터로 각 인덱스는 단어의 빈도수

CountVectorizer 활용

cv = CountVectorizer()

train_X_cv = cv.fit_transform(train_X)

encoded_trainX = train_X_cv.toarray()

print(encoded_trainX)

print(len(encoded_trainX[0])) # 37 크기의 고정벡터

print(encoded_trainX[0])

첫 문장 인코딩(변환) 데이터 :

[0 1 0 0 0 0 0 0 0 0 0 0 0 2 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0]

```
# 벡터로 인코딩된 문장에 어떤 단어가 포함됐는지 확인
# inverse_transform() : 2차원 배열 기대
print(cv.inverse_transform(encoded_trainX[0].reshape(1,-1)))
```

[array(['again', 'great', 'is', 'movie', 'this', 'watch', 'will'],
 dtype='<U9')]

```
# 벡터 37개 인덱스가 의미하는 단어 확인
print(cv.get_feature_names())
```

```
# 다항분포 나이브베이즈 모델 학습
mnb = MultinomialNB()
train_Y = train_Y.astype('int')
mnb.fit(train_X_cv, train_Y)
```

['actors', 'again', 'also', 'amazing', 'and', 'awesome',
'boring', 'boyfriend', 'cool', 'director', 'do', 'ever',
'from', 'great',
'in', 'is', 'like', 'money', 'move', 'movie', 'my',
'never', 'not', 'of', 'on', 'regret',
'said', 'shame', 'sleeping', 'the', 'this', 'time',
'wasted', 'watch', 'what', 'will', 'year']

모델평가를 위한 테스트 데이터 준비

```
test_review_list = [  
    {'movie_review': 'great great great movie ever', 'type':'positive'},  
    {'movie_review': 'I like this amazing movie', 'type':'positive'},  
    {'movie_review': 'my boyfriend said great movie ever', 'type':'positive'},  
    {'movie_review': 'cool cool cool', 'type':'positive'},  
    {'movie_review': 'awesome boyfriend said cool movie ever', 'type':'positive'},  
    {'movie_review': 'shame shame shame', 'type':'negative'},  
    {'movie_review': 'awesome director shame movie boring movie', 'type':'negative'},  
    {'movie_review': 'do not like this movie', 'type':'negative'},  
    {'movie_review': 'I do not like this boring movie', 'type':'negative'},  
    {'movie_review': 'aweful terrible boring movie', 'type':'negative'},  
]
```

```
test_review_df = pd.DataFrame(test_review_list)  
test_review_df['label'] = test_review_df['type'].map({'positive':1,'negative':0})  
print(test_review_df)
```

```
test_review_X = test_review_df['movie_review']  
test_review_Y = test_review_df['label']
```

test 데이터 변환 진행

```
test_reviewX_cv = cv.transform(test_review_X)
test_predicted = mnbc.predict(test_reviewX_cv)
print(test_predicted)
```

[1 1 1 1 1 0 0 0 0]

```
print('accuracy : ', accuracy_score(test_review_Y, test_predicted)) # accuracy : 1.0
```

#임의의 영화 감상평 예측하기 위한 인코딩(변환)

```
tempcv = cv.transform(["It's boring and uninteresting movie",
                        'cool cool great',
                        "This is a very interesting and fun movie",
                        'i will never watch this movie again and i will not like it'])
```

```
temp_predicted = mnbc.predict(tempcv)
print(temp_predicted)
```

[0 1 1 0]

```
movie_review_eval = ['부정','긍정']
```

```
movie_review_evalarray = np.array(movie_review_eval)
```

```
print(movie_review_evalarray[temp_predicted])
```

['부정' '긍정' '긍정' '부정']

Thank You